转向平衡车 LQR 控制系统技术实现文档

> 基于 Cornell 点质量模型的嵌入式 LQR 平衡控制器设计与实现
>
> 作者: 无心之举

1. 系统概述

1.1 平衡原理

转向平衡车（摩托车结构）的平衡基于**离心力恢复**机制:

```
左倾 → 左转 → 离心力右推 → 回正
右倾 → 右转 → 离心力左推 → 回正
```

Fence 1

与四轮车辆不同，单轨车辆必须通过主动转向产生的离心力来维持平衡。LQR（线性二次型调节器）通过优化转向策略，在保持平衡的同时最小化控制能量。

1.2 控制架构

系统采用分层架构:

```
┌──────────────────────────────────────────────────────────┐
│                        应用层 (lqr_balance.c)              │
│  • IMU数据获取与坐标变换                                    │
│  • 速度估计 (电机RPM → m/s)                                 │
│  • 舵机PWM映射                                              │
│  • 压弯控制 (期望侧倾角设定)                                 │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                       驱动层 (lqr_driver.c)                │
│  • LQR状态反馈计算: u = -K * x                              │
│  • 增益调度 (速度查表 + 线性插值)                            │
│  • 转向角积分与限幅                                         │
│  • 低速保护逻辑                                             │
└──────────────────────────────────────────────────────────┘
```

Fence 2

2. 数学模型

2.1 Cornell 点质量模型

本实现采用 Cornell University 自动驾驶自行车项目的简化模型，将复杂的多体动力学简化为三阶系统。

2.1.1 动力学方程

侧倾角加速度由两项主导：

$\ddot{\phi} = \frac{g}{h}\phi - \frac{v^2}{hL}\delta - \frac{bv}{hL}\dot{\delta}$

符号	含义	物理意义
ϕ	侧倾角 (rad)	车身左右倾斜角度
δ	转向角 (rad)	车把/前轮偏转角度
h	质心高度 (m)	决定倒立摆不稳定性
L	轴距 (m)	影响转弯半径和离心力
b	拖曳距 (m)	惯性耦合项系数
v	速度 (m/s)	离心力与速度平方成正比
g	重力加速度	9.8 m/s^2

Table 1

2.2 状态空间表示

状态向量: $\mathbf{x} = [\phi, \dot{\phi}, \delta]^T$

控制输入: $u = \dot{\delta}$ （转向角速度）

连续时间状态空间:

$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u$

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{g}{h} & 0 & -\frac{v^2}{hL} \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ -\frac{bv}{hL} \\ 1 \end{bmatrix}$$

矩阵元素物理意义:

元素	表达式	意义
A_{21}	g/h	**不稳定性因子**，正值表示开环不稳定
A_{23}	$-v^2/hL$	**离心力控制效能**，速度越快控制越有效
B_2	$-bv/hL$	**惯性耦合**，转向加速度对侧倾的即时影响

Table 2

2.3 LQR 代价函数

$J = \int_0^\infty (\mathbf{x}^T\mathbf{Q}\mathbf{x} + u^T R \cdot u)\, dt$

权重矩阵选取（本实现使用的参数）：

$$\mathbf{Q} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 30 \end{bmatrix}, \quad R = 2.0$$

权重	值	调参效果
Q_{11}	100	强惩罚侧倾角偏差，优先保持直立
Q_{22}	15	适度惩罚侧倾速度，增加阻尼减少振荡
Q_{33}	30	惩罚转向角偏差，促进直线行驶
R	2.0	控制平滑度，防止舵机抖动

Table 3

3. 代码实现详解

3.1 数据结构

```c
// lqr_driver.h

// 物理参数
typedef struct {
    float h;    // 质心高度 (m)
    float L;    // 轴距 (m)
    float b;    // 拖曳距 (m)
    float g;    // 重力加速度
} LQR_PhysicalParams_t;

// LQR状态
typedef struct {
    float phi;      // 侧倾角 (rad)
    float phi_dot;  // 侧倾角速度 (rad/s)
    float delta;    // 转向角 (rad)
} LQR_State_t;

// 增益表条目 (用于增益调度)
typedef struct {
    float velocity;                 // 速度点 (m/s)
    float K[LQR_STATE_DIM];         // [K_phi, K_phi_dot, K_delta]
} LQR_GainEntry_t;
```

Fence 3

3.2 增益调度实现

由于 $\mathbf{A}$ 矩阵依赖速度 v，不同速度下的最优增益 K 不同。本实现采用**查表 + 线性插值**方式：

```c
// lqr_driver.c - 预计算增益表
static const LQR_GainEntry_t default_gain_entries[] = {
    {0.5f,  {-214.7704f, -23.9102f, 22.1226f}},  // 低速: 高增益
    {1.0f,  {-64.0971f,  -7.5628f,  23.9397f}},
    {1.4f,  {-36.5193f,  -4.8185f,  28.2038f}},
    // ... 更多速度点
    {7.0f,  {-11.5509f,  -2.9192f, 106.6175f}},  // 高速: 低增益
};
```

观察规律:

Fence 4

- **K_phi（侧倾增益）**：低速时绝对值大（-214），高速时小（-11），因为低速下离心力弱，需要更激进的转向
- **K_delta（转向增益）**：随速度增加而增加，高速时需要更强的回正力

插值算法:

```c
int8_t lqr_update_gain(LQR_Controller_t *ctrl, float velocity)
{
    // 低速保护
    if (velocity < ctrl->v_min) {
        ctrl->gain_valid = 0;
        return -1;
    }

    // 查找插值区间
    for (i = 0; i < table->size - 1; i++) {
        if (velocity >= table->entries[i].velocity &&
            velocity < table->entries[i + 1].velocity) {
            idx = i;
            break;
        }
    }

    // 线性插值
    t = (velocity - v1) / (v2 - v1);
    for (i = 0; i < LQR_STATE_DIM; i++) {
        ctrl->K[i] = k1 + t * (k2 - k1);
    }
}
```

Fence 5

3.3 核心控制律

```c
// lqr_driver.c
float lqr_compute(LQR_Controller_t *ctrl, float phi, float phi_dot)
{
    // 更新状态
    ctrl->state.phi = phi;
    ctrl->state.phi_dot = phi_dot;
    ctrl->state.delta = ctrl->delta_cmd;

    // LQR状态反馈: u = -K * (x - x_target)
    // u 是转向角速度 delta_dot
    float error_delta = ctrl->state.delta - ctrl->target_delta;
    u = -(ctrl->K[0] * phi +
          ctrl->K[1] * phi_dot +
          ctrl->K[2] * error_delta);

    // 转向角速度限幅
    u = lqr_constrain_float(u, -ctrl->delta_dot_max, ctrl->delta_dot_max);

    // 积分得到转向角命令
    ctrl->delta_cmd += u * LQR_DT;  // LQR_DT = 0.005s (5ms)

    // 转向角限幅
    ctrl->delta_cmd = lqr_constrain_float(ctrl->delta_cmd,
                                          -ctrl->delta_max, ctrl->delta_max);

    return ctrl->delta_cmd;
```

```
}
```

Fence 6

关键设计决策:

1. **输入为转向角速度** $\dot{\delta}$ 而非转向角：与舵机特性匹配，动作更平滑

2. **积分产生转向角命令**: `delta_cmd += u * dt`

3. **目标转向角支持**: 允许设置 `target_delta` 实现压弯控制

3.4 坐标系对齐与单位转换

```c
// lqr_balance.c - IMU数据处理
void lqr_balance_control(void)
{
    // IMU定义: 左倾为正
    // LQR模型: 右倾为正
    // 需要取反
    phi = -(pitch_kalman - lqr_expect_phi) * DEG_TO_RAD;
    phi_dot = -gyro_y_rate * DEG_TO_RAD;
}
```

Fence 7

单位统一:

- IMU 输出：角度（度），角速度（度/秒）
- LQR 内部：弧度（rad），弧度/秒（rad/s）
- 舵机输出：PWM 脉宽

3.5 压弯控制（转弯时的稳态倾角）

利用稳态离心力平衡原理，转弯时车辆需要向内侧倾斜:

$$\tan(\phi_{target}) = \frac{v^2}{gL} \cdot \delta$$

代码实现:

```c
// lqr_balance.c
if (velocity > 0.5f) {
    lqr_ctrl.target_delta = tanf(target_phi_rad) * g * L / (velocity * velocity);
} else {
    lqr_ctrl.target_delta = 0.0f;  // 低速时不允许压弯
}
```

Fence 8

4. 参数配置说明

4.1 物理参数（需实测）

```
// lqr_balance.c
lqr_set_physical_params(&lqr_ctrl,
    0.12f,   // h: 质心高度 12cm
    0.21f,   // L: 轴距 21cm
    0.02f);  // b: 拖曳距 2cm
```

Fence 9

测量方法:

参数	测量方法
h	悬挂法：分别从前后轮吊起车辆，两垂线交点即为质心
L	直接测量前后轮轴心水平距离
b	测量转向轴延长线与地面交点到前轮触地点的距离

Table 4

4.2 控制限幅

```
// lqr_balance.c
lqr_set_limits(&lqr_ctrl,
    LQR_DELTA_MECHANICAL_MAX,  // 最大转向角 ~23°
    8.0f,                      // 最大转向角速度 rad/s
    0.3f);                     // 最小工作速度 0.3m/s

// lqr_balance.h
#define LQR_DELTA_MECHANICAL_MAX    DEG_RAD(23.0f)
```

Fence 10

4.3 舵机映射

```
// lqr_balance.h
#define LQR_SERVO_MID           mid      // 中位PWM
#define LQR_SERVO_LEFT          l_max    // 左极限
#define LQR_SERVO_RIGHT         r_max    // 右极限

// 转向角 -> PWM 映射 (支持非对称舵机)
uint32 lqr_delta_to_servo_pwm(float delta)
{
    float ratio = delta / LQR_DELTA_MECHANICAL_MAX;

    if (ratio >= 0)
        pwm = MID + ratio * (RIGHT - MID);
    else
        pwm = MID + ratio * (MID - LEFT);

    return (uint32)pwm;
}
```

Fence 11

5. 控制流程

5.1 调用时序

```
5ms 定时中断
    │
    ├─▶ lqr_balance_control()
    │       │
    │       ├─▶ lqr_get_velocity()      // 电机RPM → m/s
    │       │
    │       ├─▶ lqr_update_gain()       // 速度查表插值
    │       │
    │       ├─▶ 计算目标转向角          // 压弯控制
    │       │
    │       ├─▶ IMU数据转换             // 坐标系对齐
    │       │
    │       ├─▶ lqr_compute()           // 核心LQR计算
    │       │
    │       └─▶ servo_set()             // 舵机输出
    │
    └─▶ 其他任务
```

Fence 12

5.2 状态机

```
        ┌──────────────────┐
        │     初始化        │
        │  lqr_balance_init │
        └──────────┬───────┘
                   │
                   ▼
        ┌──────────────────┐        v < v_min
        │     低速等待      │◀──────────────┐
        │  delta_cmd *= 0.95 │                │
        └──────────┬───────┘                │
                   │ v >= v_min              │
                   ▼                         │
        ┌──────────────────┐                │
        │     正常控制      │────────────────┘
        │   lqr_compute()   │      v < v_min
        └──────────────────┘
```

Fence 13

6. 增益表生成（Python）

6.1 离线计算脚本

```python
import numpy as np
from scipy import linalg

def compute_lqr_gain(h, L, b, v, Q, R):
    """计算单个速度点的LQR增益"""
    g = 9.8

    # 状态空间矩阵
    A = np.array([
        [0,   1,       0],
        [g/h, 0, -v**2/(h*L)],
        [0,   0,       0]
    ])

    B = np.array([
        [0],
        [-b*v/(h*L)],
        [1]
    ])

    # 求解 Riccati 方程
    P = linalg.solve_continuous_are(A, B, Q, R)

    # 计算增益 K = R^(-1) * B^T * P
    K = np.linalg.inv(R) @ B.T @ P

    return K.flatten()

# 物理参数（与代码一致）
h = 0.12   # 质心高度
L = 0.21   # 轴距
b = 0.02   # 拖曳距

# LQR权重
Q = np.diag([100.0, 15.0, 30.0])
R = np.array([[2.0]])

# 速度范围
velocities = np.arange(0.5, 7.5, 0.5)

# 生成增益表
print("static const LQR_GainEntry_t default_gain_entries[] = {")
for v in velocities:
    K = compute_lqr_gain(h, L, b, v, Q, R)
    print(f"    {{{v:.1f}f, {{{K[0]:.4f}f, {K[1]:.4f}f, {K[2]:.4f}f}}}},")
print("};")
```

Fence 14

6.2 验证稳定性

```python
import control

# 验证闭环极点
A_cl = A - B @ K
eigenvalues = np.linalg.eigvals(A_cl)
print(f"闭环极点: {eigenvalues}")
# 所有极点实部应为负（稳定）
```

Fence 15

7. 调试与调参指南

7.1 常见问题排查

现象	可能原因	解决方案
倒向一边不动	舵机方向反	检查 `lqr_delta_to_servo_pwm()` 映射
疯狂抖动	增益过大 / 噪声	增大 R 值或加强 IMU 滤波
反应迟钝后倒地	增益过小	减小 R 值或增大 Q_phi
能平衡但蛇行	δ 惩罚不足	增大 Q_delta
高速振荡	高速增益不当	调整增益表中高速段参数

Table 5

7.2 调参顺序建议

1. **先验证舵机方向**：手动推车倾斜，观察舵机是否转向倾斜方向
2. **低速增益**：从 2.0m/s 开始，逐步调整 K_phi
3. **阻尼调节**：增加 Q_phi_dot 减少振荡
4. **回正能力**：调整 Q_delta 改善直线行驶
5. **扩展速度范围**：验证后逐步增加速度

7.3 调试接口

```c
// 获取实时调试信息
float phi, velocity, delta_deg;
lqr_get_debug_info(&phi, &velocity, &delta_deg);

// 运行时调整参数
lqr_tune_params(0.13f, 0.22f);  // 调整 h, L
```

Fence 16

8. API 参考

8.1 应用层接口 (lqr_balance.h)

Table 6

函数	说明
`void lqr_balance_init(void)`	初始化 LQR 平衡控制
`void lqr_balance_control(void)`	主控制函数，5ms 中断调用
`void lqr_set_expect_phi(float phi_deg)`	设置期望侧倾角（度），用于转弯
`float lqr_get_velocity(void)`	获取当前速度估计
`void lqr_tune_params(float h, float L)`	运行时调整物理参数

8.2 驱动层接口 (lqr_driver.h)

Table 7

函数	说明
`void lqr_init(LQR_Controller_t *ctrl)`	初始化控制器结构体
`void lqr_set_physical_params(...)`	设置物理参数
`int8_t lqr_update_gain(..., float v)`	根据速度更新增益
`float lqr_compute(..., float phi, float phi_dot)`	执行 LQR 计算
`void lqr_enable(LQR_Controller_t *ctrl, uint8_t en)`	使能/禁用控制器
`void lqr_reset(LQR_Controller_t *ctrl)`	重置控制器状态